If you now look at failure spans by filtering for the HTTP status code (in the `tags` input field, add, for example, `501` or filter by `error=true`) or clicking on one of the spans that shows an error (you should see a red exclamation mark on that span), you should see something like what is shown in figure 8.7. What does this mean? Jaeger provided us with a visual rendering of failures and allowed us to drill down into services that show errors. By clicking on a span, you can explore the service and compare things like execution time duration with the baseline you have from the happy path as well as see what the error code is. This means you can pinpoint services that cause issues very quickly and then focus on those to troubleshoot.

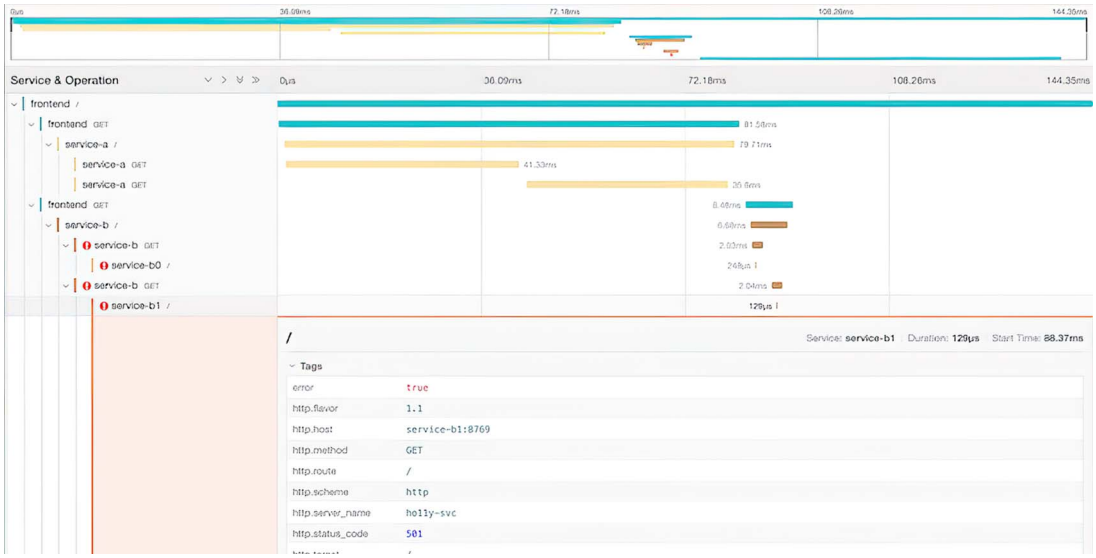


Figure 8.7 Troubleshooting the demo microservices app: an example failure trace and the span that caused the failure

This hands-on exercise using distributed tracing for troubleshooting a microservices-based app is now complete. Next, we will discuss how to benefit from our new app in a practical setup.

8.3 Practical considerations

Now that you know the basics of distributed tracing and have seen it in action in the context of a concrete use case, let's talk a little bit more about things to consider when going all in with distributed tracing. On a high level, it's important to understand that not every span of a trace has the same information value. For example, if 99 out of 100 times you see a span with an HTTP status code of `200`, what does this tell you? In contrast, the one span that carries a `500` HTTP status code indicates that the service that emitted the span likely had some kind of issue you may wish to research deeper.